UF Herbert Wertheim College of Engineering
UNIVERSITY of FLORIDA

Electrical and Computer Engineering

Microcontroller ADC + LCD Module (Ohm meter)

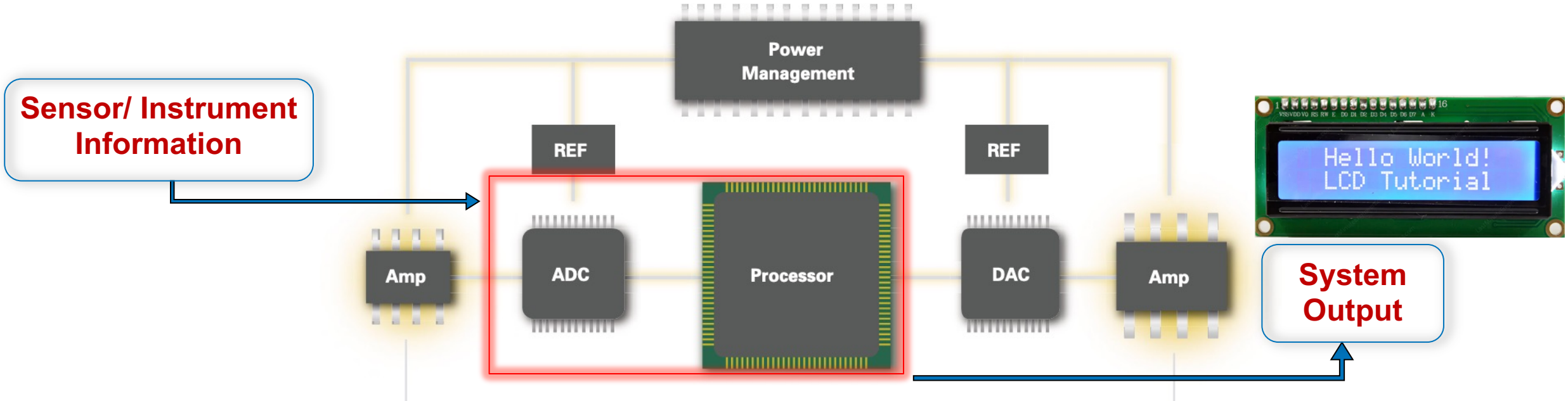
EEL3923C Design 1

Erin Patrick, Mike Stapleton, Eric Liebner

POWERING THE NEW ENGINEER TO TRANSFORM THE FUTURE

ADC + LCD Module Objectives

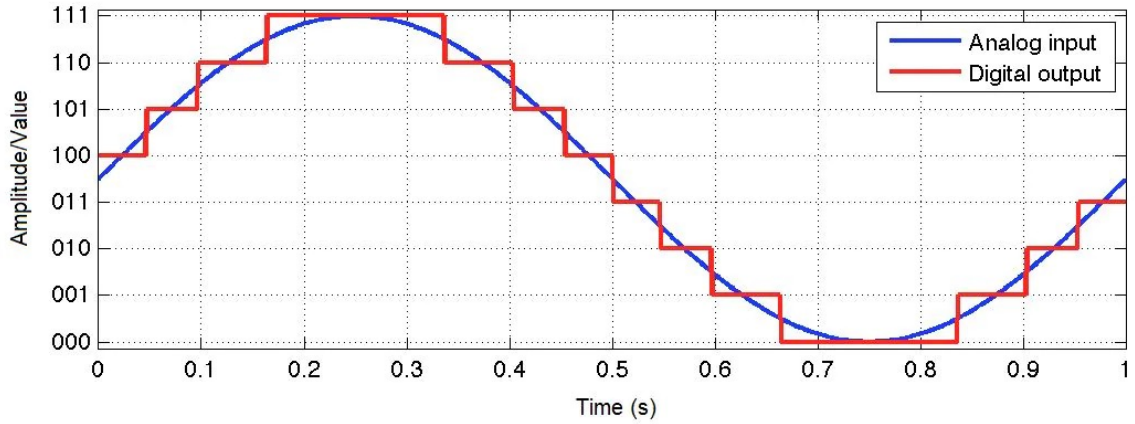
- Learn how to use the built-in analog to digital converter in the microcontroller
  - Understand its limitations
- Use parallel communication to interface with an LCD



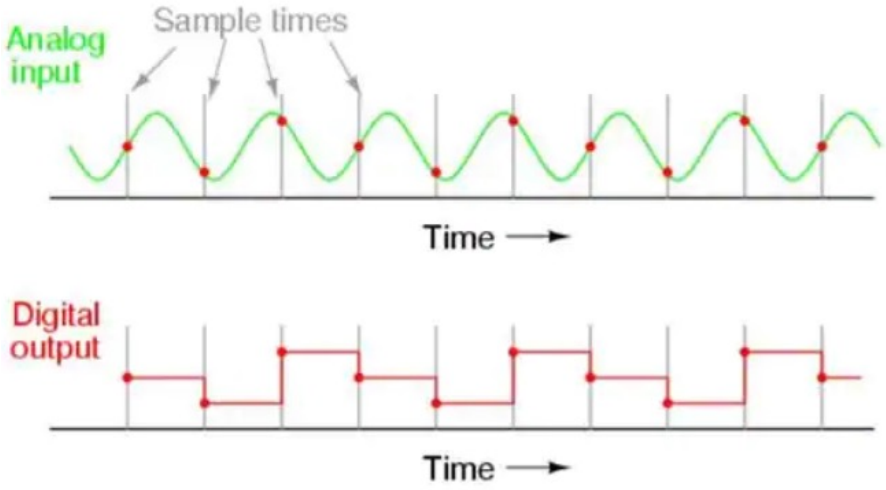
Analog-to Digital Converters (ADC): Fundamentals

An ADC will convert a continuous time signal into a discrete one by measuring it at a defined frequency

- Shannon's Theorem:
  - sampling frequency must be at least twice the signal frequency. If not followed, then the signal will be under-sampled, which leads to aliasing and invalid signal reconstruction
- Nyquist frequency:
  - the minimum signal frequency one can sample without aliasing (Nyquist frequency = fs/2)



An example of an ADC sampling at a proper sampling rate, with some quantization noise



An example of an ADC sampling at an improper sampling rate (Aliasing)

Analog-to Digital Converters (ADC): Fundamentals

- Span:
  - the input range of voltage over which the ADC will digitize that input (usually power rail limited, i.e. 0V - 5 V or 0 V - 3.3 V)
- Resolution:
  - the smallest increment corresponding to a 1 LSB converter code change. For converters, resolution is normally expressed in bits, where the number of digital codes is equal to 2^n (i.e. 12-bit converter: 2^{12} = 4096 digital codes)
- Accuracy:
  - = span/resolution, for example a 10-bit converter with a 0 V - 3.3 V span would have an accuracy of 3.3/1024 = 3.2 mV/bit

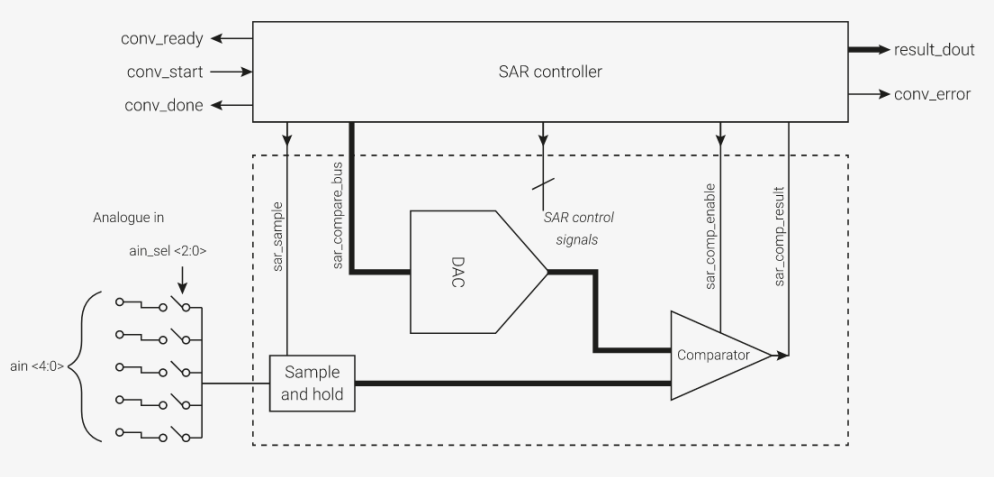
2-bit resolution	3-bit resolution
$00_2 \approx 0$ V	$000_2 \approx 0$ V
$01_2 \approx 1.667$ V	$001_2 \approx 0.714$ V
$10_2 \approx 3.334$ V	$010_2 \approx 1.428$ V
$11_2 \approx 5$ V	$011_2 \approx 2.142$ V
	$100_2 \approx 2.856$ V
	$101_2 \approx 3.570$ V
	$110_2 \approx 4.284$ V
	$111_2 \approx 4.998$ V

For Pico: 12 bit ADC, 3.3 V supply rail

- accuracy = 3.3 V / 2^{12} bits = 0.806 mV/bit (ideal)
- Error sources make the real accuracy lower
  - See next slides

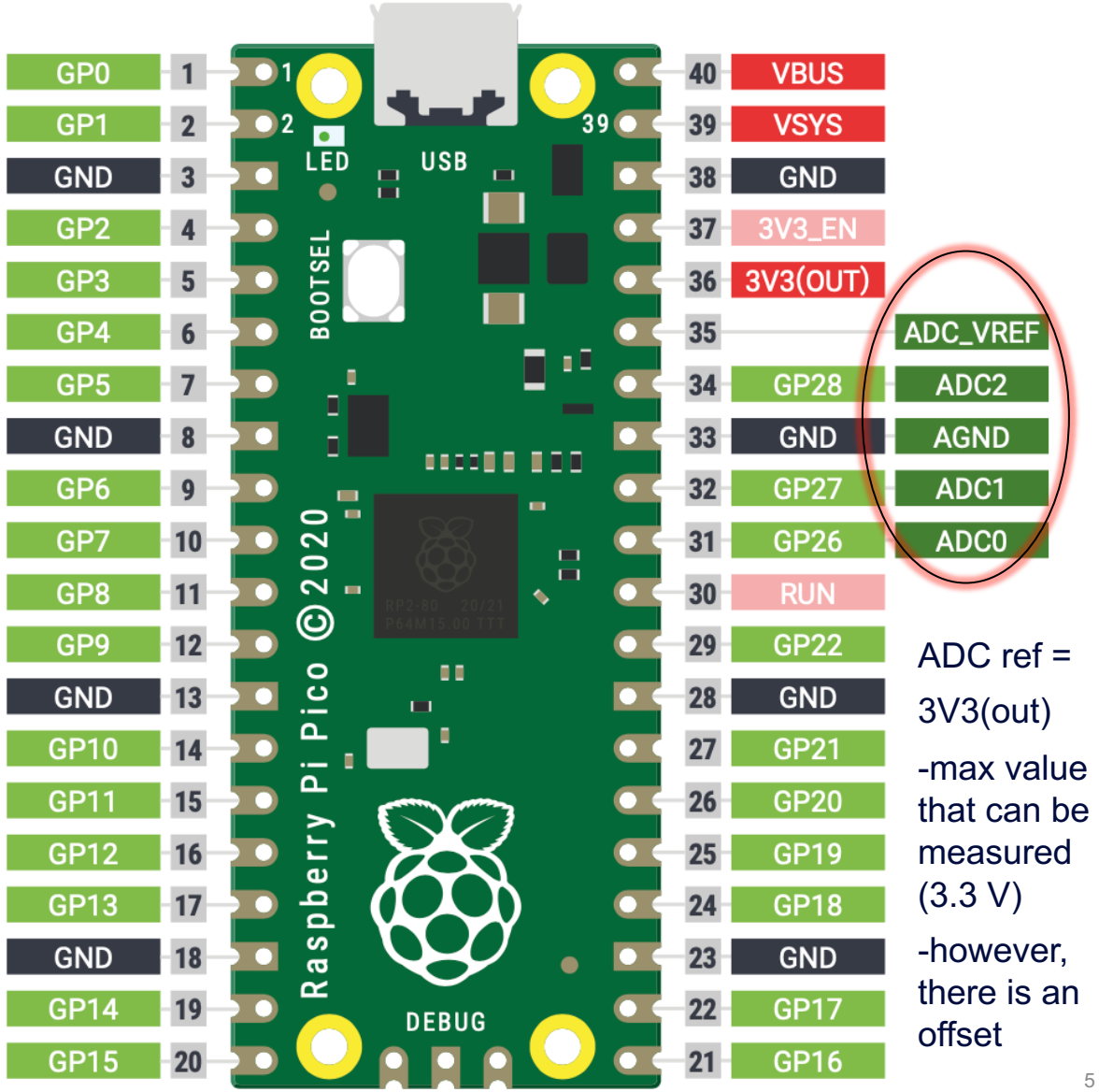
Raspberry Pi Pico ADC

- https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf
- Page 585



The SAR ADC (Successive Approximation Register Analog to Digital Converter) is a combination of digital controller and analog circuit.

- The ADC requires a 48 MHz clock; 96 clock cycles to capture a sample : 96/48 MHz = 2 µs per sample (500 kHz sampling rate)
- 12 bit ADC



ADC ref = 3V3(out)

-max value that can be measured (3.3 V)

-however, there is an offset

Analog to Digital Converters (ADC): Error and Noise

Quantization Noise

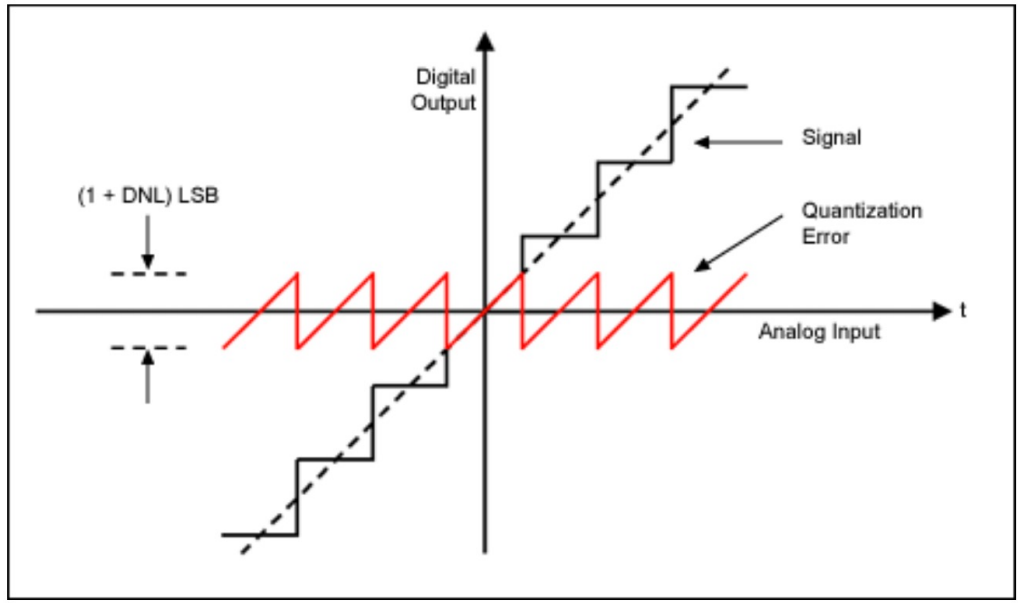


Figure 1. Quantization noise or error is the center saw-tooth waveform; it is the residual left when the signal dotted line is quantized into digital steps

Source: https://www.maximintegrated.com/en/design/technical-documents/tutorials/5/5061.html

Input Referred Noise (thermal and device noise)

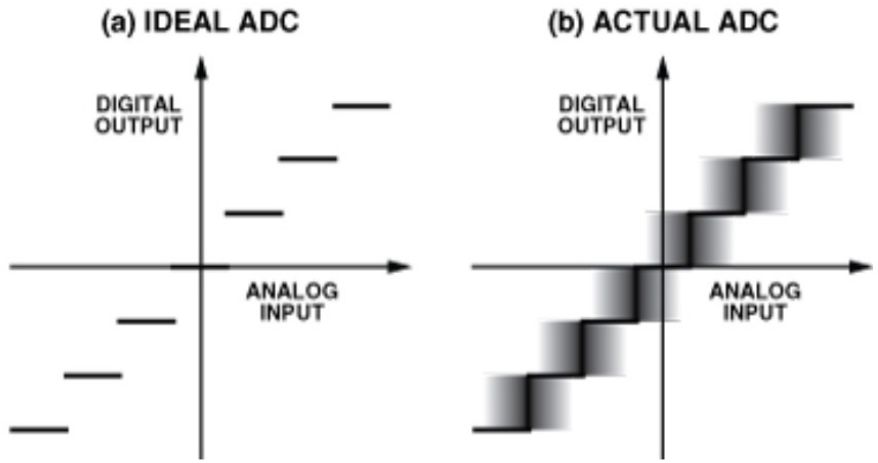


Figure 1. Code-transition noise (input-referred noise) and its effect on ADC transfer function.

Source: https://www.analog.com/en/analog-dialogue/articles/adc-input-noise.html

Analog to Digital Converters (ADC): Offset Error

The Pico has significant offset error

- Read Pico data sheet to figure out ways to mitigate it

https://datasheets.raspberrypi.com/pico/pico-datasheet.pdf

Read Data sheet p.17-18

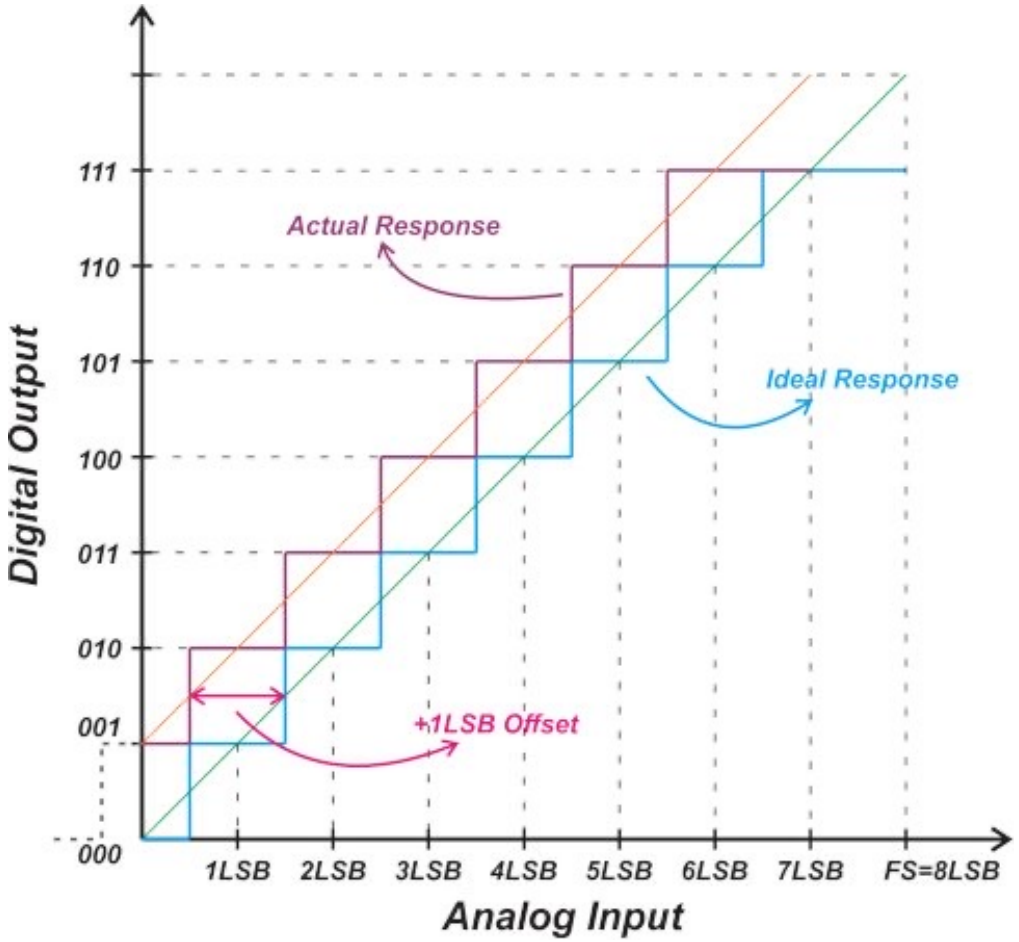


Image source: https://www.allaboutcircuits.com/technical-articles/adc-offset-and-gain-error-specifications/

Liquid Crystal Display (LCD)

CFAH1602Z-YYH-ET (in lab kit)

- 16 character, 2 line
- uses ST7066U controller/driver
- We will communicate to it via GPIO lines on the MCU
  - In a parallel fashion
  - Using “bit-banging” method to provide control information
  - This method is GPIO expensive

Compare to an LCD with serial communication hardware

- One can run SPI or I2C protocols on designated MCU hardware
- On very few pins

LCD Parallel Communication

Read Data sheet for LCD controller: https://www.crystalfontz.com/controllers/Sitronix/ST7066U/499/

6. Interface Pin Function

Pin No.	Symbol	Level	Function
1	V_{SS}	0v	Ground
2	V_{DD}	5.0v	Supply Voltage for Logic
3	V_O	(variable)	Supply Voltage for LCD
4	RS	H/L	H: Data L: Instruction Code
5	R/W	H/L	H: Read L: Write
6	E	H, H → L	Chip Enable Signal
7	DB0	H/L	Data Bus Line
8	DB1	H/L	Data Bus Line
9	DB2	H/L	Data Bus Line
10	DB3	H/L	Data Bus Line
11	DB4	H/L	Data Bus Line
12	DB5	H/L	Data Bus Line
13	DB6	H/L	Data Bus Line
14	DB7	H/L	Data Bus Line
15	LED (+)		Anode of LED Backlight
16	LED (-)		Cathode of LED Backlight

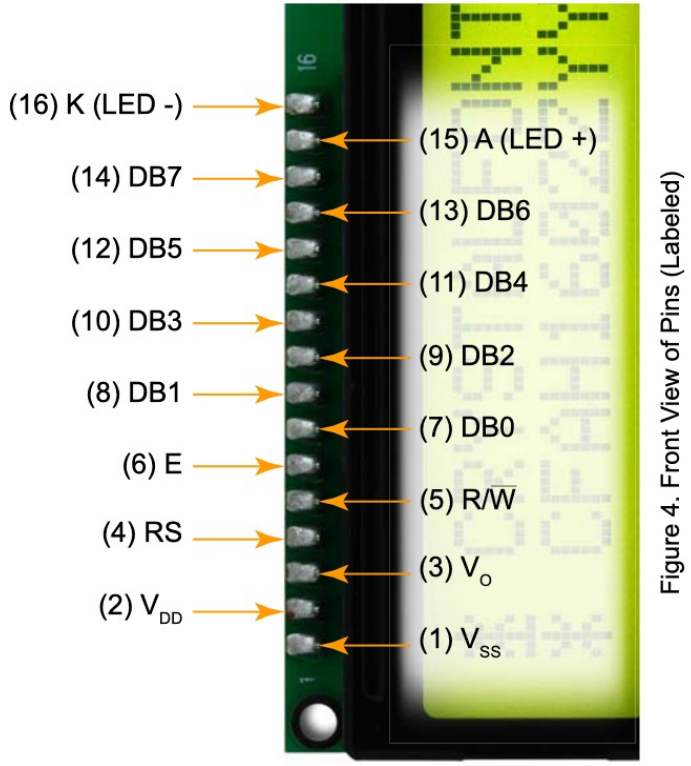


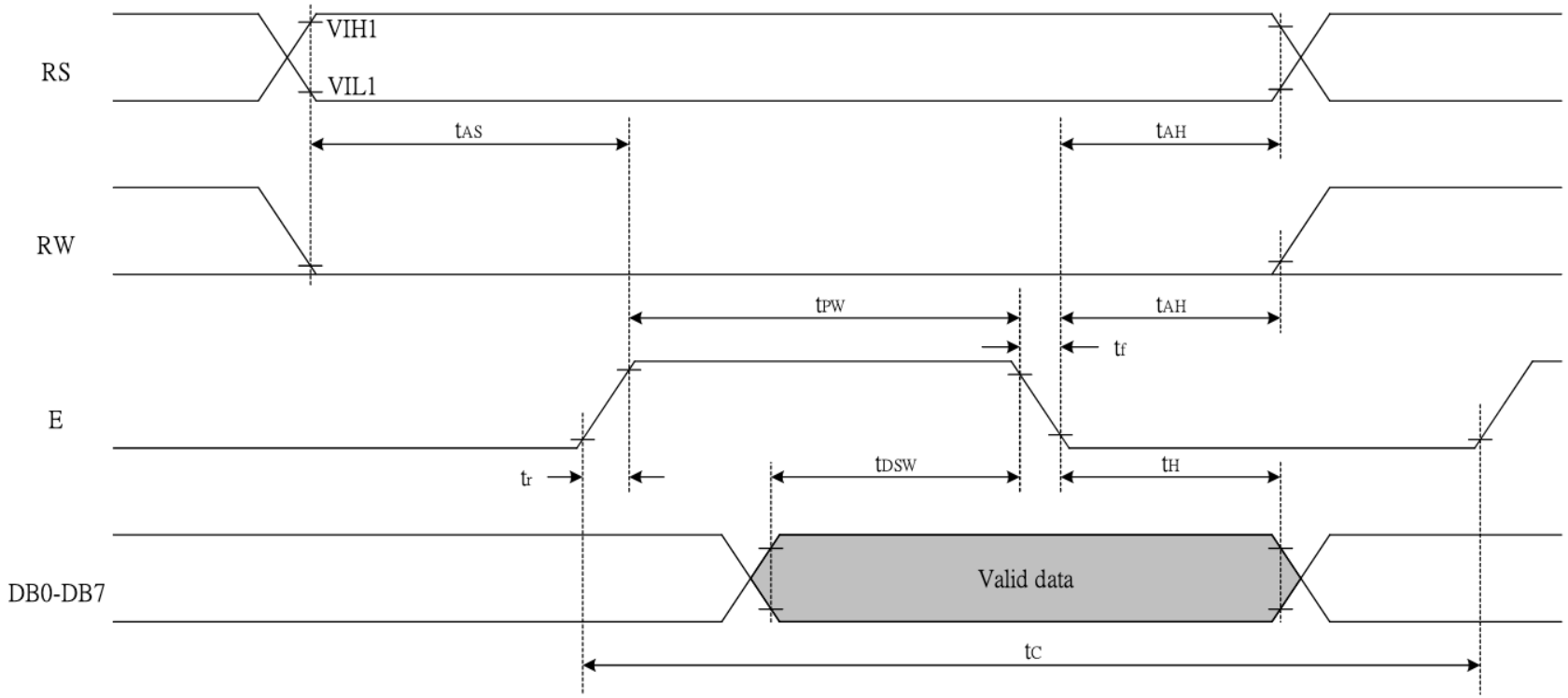
Figure 4. Front View of Pins (Labeled)

We will only be writing 4 bits at once

LCD Parallel Communication

From ST7066U data sheet, page 33

- **Writing data from MPU to ST7066U**



Write sequence:

RW can stay low
RS – high
E – high
D4-7 - data bits
E – low
RS – low

Sample Pico Code

```
#-----------------------------------------------
#          PARALLEL LCD FUNCTIONS
#          ======================
# These functions initialize and control the LCD
#
# Author: Dogan Ibrahim
# File  : LCD
# Date  : August, 2021
#-----------------------------------------------
from machine import Pin
import utime

EN = Pin(16, Pin.OUT)
RS = Pin(17, Pin.OUT)
D4 = Pin(18, Pin.OUT)
D5 = Pin(19, Pin.OUT)
D6 = Pin(20, Pin.OUT)
D7 = Pin(21, Pin.OUT)
```

Configure GPIO pins (you may use any GPIO pins on the Pico)

```
PORT = [18, 19, 20, 21]
L = [0,0,0,0]

def Configure():
   for i in range(4):
      L[i] = Pin(PORT[i], Pin.OUT)

def lcd_strobe():
   EN.value(1)
   utime.sleep_ms(1)
   EN.value(0)
   utime.sleep_ms(1)
```

Sample Pico Code

```
def lcd_write(c, mode):
    if mode == 0:
        d = c
    else:
        d = ord(c)
    d = d >> 4
    for i in range(4):
        b = d & 1
        L[i].value(b)
        d = d >> 1
    RS.value(mode)
    lcd_strobe()
```

Send upper nibble of 8 bits

```
    if mode == 0:
        d = c
    else:
        d = ord(c)
    for i in range(4):
        b = d & 1
        L[i].value(b)
        d = d >> 1
    RS.value(mode)
    lcd_strobe()
    utime.sleep_ms(1)
    RS.value(1)
```

Send lower nibble of 8 bits

Functions to send instruction codes

```
def lcd_clear():
    lcd_write(0x01, 0)
    utime.sleep_ms(5)

def lcd_home():
    lcd_write(0x02, 0)
    utime.sleep_ms(5)

def lcd_cursor_blink():
    lcd_write(0x0D, 0)
    utime.sleep_ms(1)

def lcd_cursor_on():
    lcd_write(0x0E, 0)
    utime.sleep_ms(1)

def lcd_cursor_off():
    lcd_write(0x0C, 0)
    utime.sleep_ms(1)
```

Example: c = “P” -> 80 (integer representing Unicode character; ord() does this conversion)

80 = hex 50 = binary 01010000

Upper nibble Lower nibble

Sample Pico Code

```
def lcd_puts(s):
    l = len(s)
    for i in range(l):
        lcd_putch(s[i])

def lcd_putch(c):
    lcd_write(c, 1)
```

Use this function to write a string

Use this function to write a single character

```
def lcd_goto(col, row):
    c = col + 1
    if row == 0:
        address = 0
    if row == 1:
        address = 0x40
    if row == 2:
        address = 0x14
    if row == 3:
        address = 0x54
    address = address + c - 1
    lcd_write(0x80 | address, 0)
```

Initialize LCD

```
def lcd_init():
    Configure()
    utime.sleep_ms(120)
    for i in range(4):
        L[i].value(0)
    utime.sleep_ms(50)
    L[0].value(1)
    L[1].value(1)
    lcd_strobe()
    utime.sleep_ms(10)
    lcd_strobe()
    utime.sleep_ms(10)
    lcd_strobe()
    utime.sleep_ms(10)
    L[0].value(0)
    lcd_strobe()
    utime.sleep_ms(5)
    lcd_write(0x28, 0)
    utime.sleep_ms(1)
    lcd_write(0x08, 0)
    utime.sleep_ms(1)
    lcd_write(0x01, 0)
    utime.sleep_ms(10)
    lcd_write(0x06, 0)
    utime.sleep_ms(5)
    lcd_write(0x0C, 0)
    utime.sleep_ms(10)
#================== END OF LCD FUNCTIONS
=======================
```

*You will run this function before the main while-true loop of your program

*This function sets up the L[] array that you will write the character data to and primes the LCD for use

```
lcd_init()
while True:
```

Microcontrollers are multifaceted!

Assignment (ADC-LCD, Ohm meter)

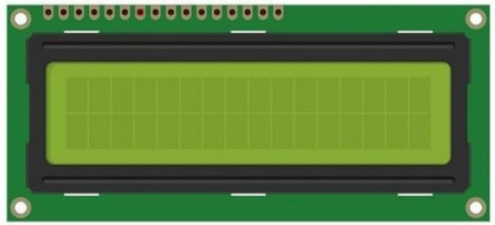
Create a microcontroller-based system that measures the resistance of a resistor between 1 kΩ and 1 MΩ using built-in ADC peripheral on MCU. Use the bit-banging method to communicate with LCD.

Requirements:

- Display the resistance in Ohms on an LCD screen (“R = XXX Ohms”)
- Display “Out of Range” if the resistance exceeds 1 MΩ or is below 1 kΩ
- Characterize the noise floor, linearity, and offset of the ADC and mitigate for accuracy using hardware and/or software techniques

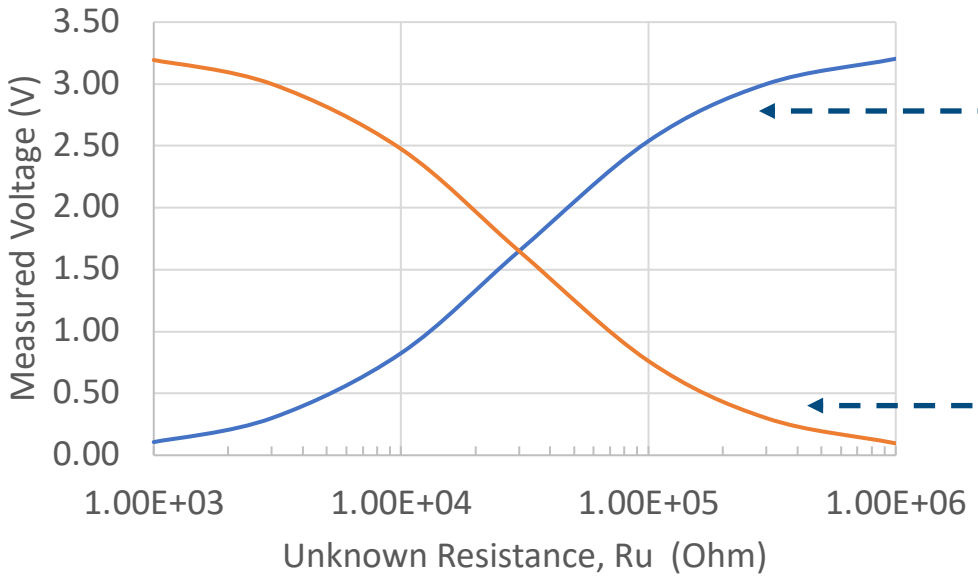
Demo:

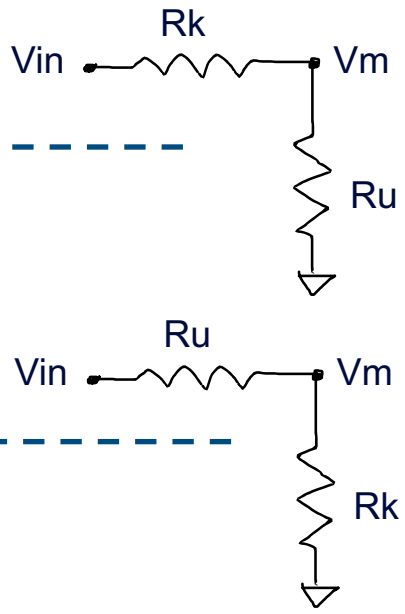
- Show working output on LCD
- Code review – explain all sections of code



Characterizing the System

Using a simple voltage divider circuit, the ideal response of the system is shown below





Vin = max voltage

Vm = measured voltage (ADC)

Rk = known resistance

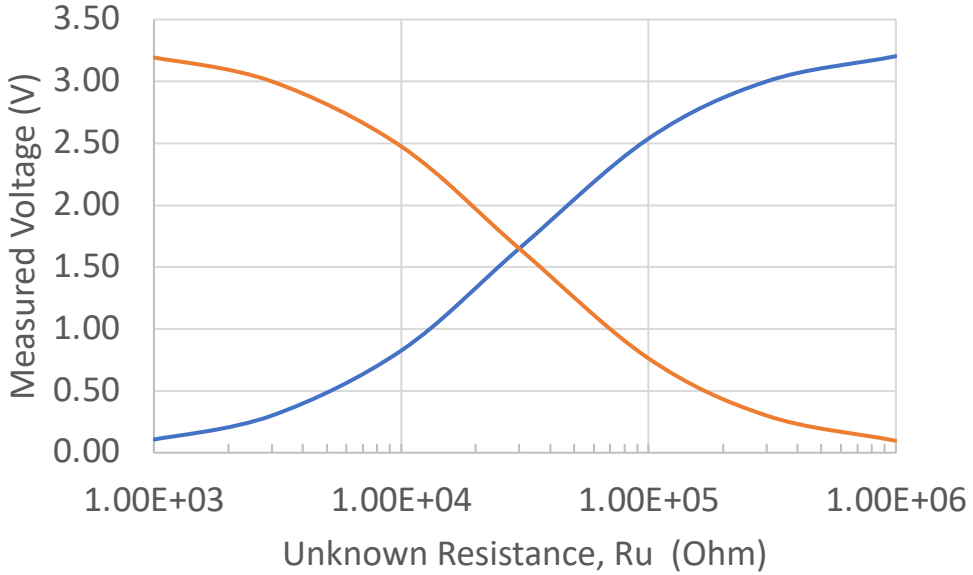
Ru = unknown resistance

* The choice of the known resistance, determines the symmetry of the response

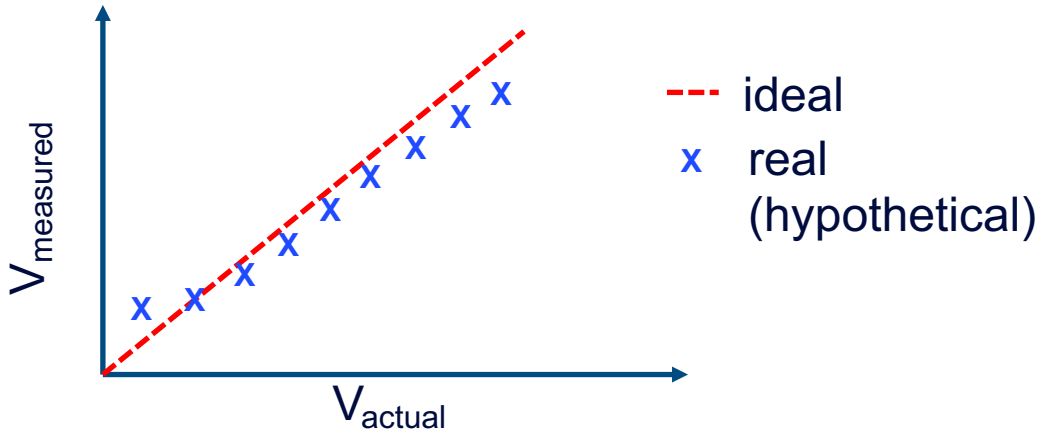
Characterizing the System

Questions to think about:

- Is the resolution of the ADC adequate for this measurement range?
- How will noise affect the measurement?
- How does ADC error/distortion/offset affect the measurement?



Determine the noise/error of system by plotting V_{actual} vs. $V_{measured}$



- Identify noise floor and errors in system
- Calibrate or redesign Ohm meter response to errors

https://datasheets.raspberrypi.com/pico/pico-datasheet.pdf
Ch 4.3. p17: good info on how to reduce ADC offset

References

Pico

- MicroPython Library Modules: https://docs.micropython.org/en/latest/rp2/quickref.html
- Pico Data sheet: https://datasheets.raspberrypi.com/pico/pico-datasheet.pdf

LDC

- Data sheet: https://www.crystalfontz.com/products/document/3776/CFAH1602Z-YYH-ETDatasheetReleaseDate2017-08-10.pdf
- Controller data sheet: https://www.crystalfontz.com/controllers/Sitronix/ST7066U/499/

How to wire it to MCU, page 6

*Don't forget a potentiometer on Vo pin